

PYTHON

Module I: Review on Lists

An essential guide to Python's fundamental data structures

Understanding Python Lists

Definition & Core Characteristics

Python lists are versatile, built-in data structures for storing ordered collections of data. They are fundamental for organizing and manipulating sequences in Python.



Ordered

Items maintain a defined sequence; new elements are added to the end.



Changeable (Mutable)

Elements can be modified, added, or removed after list creation.



Allows Duplicates

A list can contain multiple instances of the same value.



Indexed

Each item has a numerical index starting from 0 for direct



Creating Python Lists

Lists are defined using square brackets `[]` and can hold diverse data types. Python offers several intuitive methods for list initialization.



Literal Assignment

The most direct method, enclosing elements in square brackets.

```
my_list = ["apple", "banana", "cherry"]
```



`list()` Constructor

Creates a list from an existing iterable like a tuple or string.

```
another_list = list(("red", "green", "blue"))
```



List Comprehension

A concise syntax to create lists by transforming or filtering iterables.

```
squares = [x**2 for x in range(5)]
```

Essential List Functions

Common Built-in Functions

Quickly get insights into list contents.

len() Returns the number of items.

min()/max() Finds the smallest/largest item.

sum() Calculates the sum of all numeric items.

Modifying Lists

Methods to add elements and expand a list.

append(item) Adds an item to the end of the list.

insert(index, item) Inserts an item at a specified position.

extend(list) Appends elements from another list.

Removing Elements

Various ways to precisely remove items.

remove(value) Removes the first occurrence of a value.

pop(index) Removes and returns item at an index.

clear() Removes all items from the list.

del list[index] Deletes an item by index or slice.

Sorting & Reversing

Functions to reorder list elements.

sort() Sorts the list in-place (modifies original).

sorted(iterable) Returns a new sorted list.

List Comprehension: A Powerful Tool

💡 What is List Comprehension?

List comprehension provides a concise and readable way to create lists in Python. It's a hallmark of "Pythonic" code, allowing you to build a new list by applying an expression to each item in an iterable and optionally filtering elements with a condition.



<> 1. Basic Syntax & Transformation

The fundamental structure for creating new lists by transforming elements from an iterable.

SYNTAX

```
[expression for item in iterable]
```

EXAMPLE

```
# Create a list of squares
numbers = [1, 2, 3, 4]
squares = [x**2 for x in numbers]
# Result: [1, 4, 9, 16]
```

🔹 2. Adding 'if' for Filtering

Filter elements based on a condition before including them in the new list.

SYNTAX

```
[expression for item in iterable if condition]
```

EXAMPLE

```
# Select only even numbers
numbers = [1, 2, 3, 4, 5, 6]
evens = [x for x in numbers if x % 2 == 0]
# Result: [2, 4, 6]
```

📦 3. Handling Nested Scenarios

Used for creating lists from nested iterables, like flattening a list of lists.

SYNTAX

```
[expr for sub in iter for item in sub]
```

EXAMPLE

```
# Flatten a matrix
matrix = [[1, 2], [3, 4]]
flat = [num for row in matrix for num in row]
# Result: [1, 2, 3, 4]
```

★ Why Choose List Comprehensions?

Key Benefits

Comparison Example

Introduction to Tuples

{ } Defining Tuples

Tuples are a fundamental, built-in data structure in Python, designed for storing collections of items. They are ordered sequences, meaning elements maintain a defined position and can be accessed by index. Their defining characteristic is **immutability**: once a tuple is created, its elements cannot be modified, added, or removed.

☰ Ordered

Elements maintain a specific order, accessible via index (e.g., `my_tuple[0]`).

🔒 Immutable

Cannot be changed after creation, ensuring data integrity.

🔄 Heterogeneous

Can store items of different data types (int, string, etc.).

<> Syntax

Defined using parentheses `()` and commas: `(1, 'a', 2.5)`.

↔ Key Differences from Lists

Characteristic	Lists <code>[]</code>	Tuples <code>()</code>
Mutability	Mutable (Changeable)	Immutable
Performance	Slightly Slower	Slightly Faster
Memory	More Memory	Less Memory
Use Case	Dynamic data	Fixed data, records



↔ Tuple Packing & Unpacking

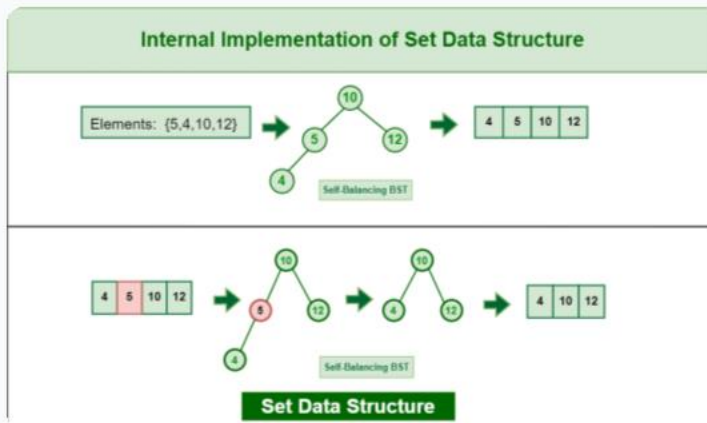
👍 When to Use Tuples

Working with Sets

6

Understanding Sets

Sets are a fundamental Python data structure for storing collections of unique, unordered items, enabling efficient membership tests and mathematical operations.



⚡ Unordered

Elements have no specific index or order.

≡ Unique

Duplicate values are automatically discarded.

✏ Mutable

Regular sets can be modified after creation.

🚫 No Indexing

Cannot access elements using an index.

Creating & Modifying

Creating Sets:

```
# Literal
my_set = {1, 2, 3}

# From list (removes duplicates)
unique = set([1, 2, 2, 3]) # -> {1, 2, 3}

# Empty set (must use set())
empty = set()
```

Modification Methods:

-  **add(el):** Adds an element. `my_set.add(4)`
-  **remove(el):** Removes element; raises `KeyError` if not found. `my_set.remove(2)`
-  **discard(el):** Removes element; no error if not found. `my_set.discard(5)`
-  **pop():** Removes and returns an arbitrary element.
-  **clear():** Removes all elements from the set.

➡ Set Operations

Venn Diagram Slide

Not everyone falls into success with their first try.



Union (|): Elements in either set.

`set1 | set2 # {1, 2, 3, 4, 5}`

Intersection (&): Elements in both sets.

`set1 & set2 # {3}`

Difference (-): In first set, not second.

`set1 - set2 # {1, 2}`

Symmetric Diff (^): In either, not both.

`set1 ^ set2 # {1, 2, 4, 5}`

Deep Dive: Frozensets (Immutable Sets)

Key Takeaways



Core Python Data Structures: At a Glance



Lists

Dynamic, mutable, general-purpose sequences. Perfect for ordered collections that may change.

Dynamic

Mutable

Ordered



Tuples

Immutable sequences for fixed collections, ensuring data integrity and efficiency for static data.

Immutable

Fixed

Ordered



Sets

Unordered collections of unique items. Ideal for membership tests and mathematical set operations.

Unique Items

Unordered

```
1111 csv_path = bpath.abspath(draw_dat.csv_export_path)
1112 post = paya.settings
1113 particles = [p for p in pays.particles] if post.type == 'ACTIVE'
1114 filenames = []
1115 if post.render_type == 'OBJRACE':
1116     dupli_ob = post.dupli_objects
1117     if dupli_ob in obj_names and draw_dat.instance_write_dupli:
1118         filepath = bpath.abspath(draw_dat.path).dupli_ob_name
1119         if os.path.exists(bpath.abspath(draw_dat.instance_export_path)):
1120             filepath = bpath.abspath(draw_dat.instance_export_path).dupli_ob_name
1121         filepath = ".join(filepath)
1122         dupli_world = dupli_ob.matrix_world.copy()
1123         trans1_inv = Matrix.Translation(-dupli_world.translation)
1124         dupli_ob.matrix_world = trans1_inv * dupli_ob.matrix_world
1125         filenames.extend(writeDupliObjects(scene, [dupli_ob], filepath, temp))
1126         dupli_ob.matrix_world = dupli_world
1127         obj.matrix_world = Matrix.identity(4)
1128         writeObject(context, instance_filepath, [obj])
1129         obj.matrix_world = obj.world
1130     else:
1131         warnings.warn("Invalid datatype '%s' for post.render_type" % post.render_type)
1132     return
1133 try:
1134     csv_file = csv_path + pays.name + ".csv" if not temp else csv_path + "csv.temp"
1135     fh = open(csv_file, "w")
1136     for p in particles:
1137         rot = Quaternion.from_Matrix(p.rotation).to_4x4()
1138         if (post.type == 'HAIR'):
1139             h1 = p.hair_keys[0].co
1140             h2 = p.hair_keys[-1].co
1141             loc = Matrix.Translation(h1)
1142             scale = Matrix.Scale((h2 - h1).length, 4)
1143             rot = emitter.matrix_world.decompose()[3].to_matrix().inverted() * rot
1144         else:
1145             loc = Matrix.Translation(p.location)
1146             scale = Matrix.Scale(p.size, 4)
1147             t = loc * rot * scale
1148             t = emitter.matrix_world * t if post.type == 'HAIR' else t
1149             writeTransform(out_t[0] * t * out_t[1], fh)
1150     fh.close()
1151     filenames.append(csv_file)
1152     return filenames
1153 except IOError as e:
1154     raise ExportException(MSG)
1155 raise ExportException(MSG)
```